Python Scikit-Learn Cheat Sheet

CORE ML WORKFLOW, PREPROCESSING & PIPELINES • @SKILL_CIRCUIT

2026 MODERN EDITION

END-TO-END MACHINE LEARNING BLUEPRINT

7-Step Lifecycle: Load → Split → Preprocess → Model Instantiation → Fit → Predict → Evaluate

Unified Estimator API

COMPLETE WORKING PIPELINE EXAMPLE

END-TO-END SNIPPET

```
from sklearn.datasets import load_iris from sklearn.model_selection import train_test_split from sklearn.preprocessing import StandardScaler from sklearn.ensemble import RandomForestClassifier from sklearn.metrics import accuracy_score, classification_report # 1. Load dataset and split with stratification X, y = load_iris(return_X_y=True) X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y) # 2. Fit scaler ONLY on train data to prevent data leakage scaler = StandardScaler() X_train_scaled = scaler.fit_transform(X_train) X_test_scaled = scaler.transform(X_test) # Transform test data using train parameters # 3. Instantiate, train, and predict model = RandomForestClassifier(n_estimators=100, random_state=42) model.fit(X_train_scaled, y_train) y_pred = model.predict(X_test_scaled) # 4. Evaluate performance print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}") print(classification_report(y_test, y_pred))
```

1. DATA LOADING & TRAIN-TEST SPLIT

INPUT DATA

Data must be 2D numeric matrices: X of shape `(n_samples, n_features)`, y of shape `(n_samples,)`. Accepts NumPy arrays or Pandas DataFrames.

```
import numpy as np import pandas as pd from sklearn.model_selection import train_test_split # Split into Train (80%) and Test (20%) X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.20, random_state=42, stratify=y ) # Time Series Split (No random shuffling!) from sklearn.model_selection import TimeSeriesSplit tscv = TimeSeriesSplit(n_splits=5)
```

✓ **stratify=y:** Crucial for classification to preserve class distribution proportions.

2. FEATURE SCALING & NORMALIZATION

NUMERIC

Essential for distance-based and gradient models (SVM, KNN, Neural Nets, Linear Models).

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler # Z-Score standardization: mean=0, std=1 scaler = StandardScaler() X_tr = scaler.fit_transform(X_train) X_te = scaler.transform(X_test) # Min-Max Scaling to [0, 1] range minmax = MinMaxScaler(feature_range=(0, 1)) # Robust Scaling (Uses median & IQR; robust to outliers!) robust = RobustScaler()
```

⚠ **Never fit scaler on test data!** Always `fit_transform(train)` then `transform(test)`.

3. IMPUTATION & CATEGORICAL ENCODERS

CLEAN & ENCODE

```
from sklearn.impute import SimpleImputer, KNNImputer from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder # Modern Imputer (replaces deprecated Imputer) imp = SimpleImputer(strategy='median') # 'mean', 'median', 'most_frequent' X_imp = imp.fit_transform(X_train) # Nominal Categorical: One-Hot Encoding ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False) X_cat_ohe = ohe.fit_transform(X_categorical) # Ordinal Features (e.g. Low, Medium, High) ord_enc = OrdinalEncoder(categories=[['Low', 'Med', 'High']])
```

4. MODERN COLUMNTRANSFORMER & PIPELINES

BEST PRACTICE

```
from sklearn.compose import ColumnTransformer from sklearn.pipeline import Pipeline from sklearn.linear_model import LogisticRegression num_cols = ['age', 'fare'] cat_cols = ['embarked', 'sex'] preprocessor = ColumnTransformer(transformers=[ ('num', Pipeline([ ('imp', SimpleImputer(strategy='median')), ('scale', StandardScaler()) ]), num_cols), ('cat', OneHotEncoder(handle_unknown='ignore'), cat_cols) ]) pipeline = Pipeline([ ('prep', preprocessor), ('clf', LogisticRegression()) ]) pipeline.fit(X_train, y_train)
```

SUPERVISED REGRESSION ESTIMATORS — CONTINUOUS

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor # Standard OLS Linear Regression lr = LinearRegression() # L2 Ridge Regression (Prevents multicollinearity) ridge = Ridge(alpha=1.0) # L1 Lasso Regression (Automated feature selection) lasso = Lasso(alpha=0.1) # Ensemble Trees rf_reg = RandomForestRegressor(n_estimators=200, max_depth=10) gb_reg = GradientBoostingRegressor(learning_rate=0.05) lr.fit(X_train, y_train) y_pred = lr.predict(X_test)
```

SUPERVISED CLASSIFICATION ESTIMATORS — DISCRETE

```
from sklearn.linear_model import LogisticRegression from sklearn.svm import SVC from sklearn.neighbors import KNeighborsClassifier from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier # Logistic Regression with L2 Regularization clf = LogisticRegression(C=1.0, max_iter=500) # Support Vector Machine with RBF Kernel svc = SVC(kernel='rbf', C=1.0, probability=True) # K-Nearest Neighbors knn = KNeighborsClassifier(n_neighbors=5, metric='minkowski') # Fast Histogram Gradient Boosting (Modern LightGBM alternative) hgb = HistGradientBoostingClassifier(max_iter=100)
```

UNSUPERVISED LEARNING: CLUSTERING & PCA — UNLABELED

```
from sklearn.cluster import KMeans, DBSCAN from sklearn.decomposition import PCA # K-Means Clustering (n_init='auto' in modern sklearn) kmeans = KMeans(n_clusters=3, n_init='auto', random_state=42) clusters = kmeans.fit_predict(X) # DBSCAN (Density-Based Clustering; handles noise/outliers) dbscan = DBSCAN(eps=0.5, min_samples=5) labels = dbscan.fit_predict(X) # PCA: Retain 95% of explained variance pca = PCA(n_components=0.95) X_reduced = pca.fit_transform(X)
```

EVALUATION METRICS & VISUAL DISPLAYS — VALIDATION

```
from sklearn.metrics import ( accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, mean_squared_error, r2_score, ConfusionMatrixDisplay ) # Classification Metrics acc = accuracy_score(y_test, y_pred) f1 = f1_score(y_test, y_pred, average='weighted') roc = roc_auc_score(y_test, model.predict_proba(X_test)[:, 1]) # Confusion Matrix Visual Display ConfusionMatrixDisplay.from_estimator(model, X_test, y_test) # Regression: Root Mean Squared Error & R2 rmse = mean_squared_error(y_test, y_pred, squared=False) r2 = r2_score(y_test, y_pred)
```

K-FOLD CROSS-VALIDATION — RELIABILITY

Never evaluate on a single train-test split. Use Stratified K-Fold for reliable validation.

```
from sklearn.model_selection import cross_val_score, StratifiedKFold, cross_validate skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42) # Multi-metric evaluation results = cross_validate( estimator=model, X=X, y=y, cv=skf, scoring= ['accuracy', 'f1_macro', 'roc_auc'], n_jobs=-1 ) print("Mean F1:", results['test_f1_macro'].mean())
```

GRIDSEARCHCV & RANDOMIZEDSEARCHCV — OPTIMIZATION

```
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV param_grid = { 'n_estimators': [100, 200, 300], 'max_depth': [None, 5, 10, 15], 'min_samples_split': [2, 5] } grid = GridSearchCV( estimator=RandomForestClassifier(random_state=42), param_grid=param_grid, cv=5, scoring='roc_auc', n_jobs=-1 ) grid.fit(X_train, y_train) print("Best Params:", grid.best_params_) best_model = grid.best_estimator_
```

THE 5 GOLDEN RULES OF SCIKIT-LEARN IN PRODUCTION

AVOID CRITICAL BUGS

Rule	Description & Why It Matters	The Solution / Modern Command
1. Zero Data Leakage	Fitting scalers or imputers on the entire dataset leaks target/variance signals from test set into train set.	Use `Pipeline([('scale', StandardScaler()), ('clf', Model())])` to automatically isolate folds.
2. Stratified Splits	Random splits on imbalanced data distort class distribution between training and evaluation.	Always pass `stratify=y` in `train_test_split()` and use `StratifiedKFold`.
3. Feature Scaling Rule	Tree-based algorithms don't need scaling, but SVM, Logistic Regression, KNN, and PCA critically fail without it.	Apply `StandardScaler()` or `RobustScaler()` for all distance/gradient estimators.
4. Handle Unknown Categories	Categorical test data might contain novel values not seen during training, triggering crash errors.	Set `OneHotEncoder(handle_unknown='ignore')` to output zero-vectors for unseen categories.
5. Model Persistence	Standard `pickle` is slow for large NumPy weight arrays and poses compatibility hazards.	Use `import joblib; joblib.dump(model, 'model.joblib')`; reload with `joblib.load()`.

Top Curated YouTube Video Courses & Channels

100% Free Video Masterclasses

▶ MATH & INTUITION

StatQuest with Josh Starmer

Channel: @statquest

The absolute gold standard for visual ML intuition. Complete playlists on Decision Trees, Random Forests, SVM, PCA, Ridge & Lasso.

Watch on YouTube →

▶ FULL MASTERCLASS

Machine Learning with Python Course

freeCodeCamp.org (10h Course)

Comprehensive end-to-end tutorial covering Scikit-Learn from scratch, supervised/unsupervised algorithms, pipelines, and real datasets.

Watch Full Course →

▶ PRODUCTION ML

CampusX – 100 Days of ML

Channel: @CampusX-official

In-depth mathematical breakdowns, custom Scikit-Learn transformers, ColumnTransformers, Pipelines, and feature engineering masterclass.

Watch Playlist →

▶ HANDS-ON PROJECTS

Krish Naik Complete ML Playlist

Channel: @krishnaik06

Industry-oriented project implementation, EDA, Scikit-Learn feature selection, hyperparameter tuning, and model deployment pipelines.

Watch Playlist →

▶ PYTHONIC CODE

Corey Schafer ML & Python Series

Channel: @coreyms

Clean coding practices, modular code structure, virtual environments, Scikit-Learn cross-validation, and clean data workflows.

Watch Tutorials →

▶ OFFICIAL TALKS

PyData & Scikit-Learn Official

Channel: @PyDataTV

Keynote talks and deep dives by Scikit-Learn core maintainers (Gael Varoquaux, Olivier Grisel) on release updates and modern architecture.

Watch Talks →

WANT MORE AI & MACHINE LEARNING ROADMAPS?

Follow **@skill_circuit** on Instagram • Daily cheatsheets, 3D architectural roadmaps & DevOps interview guides!

FOLLOW @SKILL_CIRCUIT